

CSE335 OPERATING SYSTEMS PROJECT

MINIX 3.3.0

Implementation and Native Test Report

Scheduling • Hierarchical Paging • Extent-Aware MFS Allocation

Course	Prepared by	Term	Validated
CSE335	Rodina and project team	Summer 2026	MINIX 3.3.0 VM

This concise report explains what was changed, how it was built and tested, and what the native MINIX run produced. Raw CSV files and the complete validation log are included under project-deliverables/native-results.

1. Project summary

The project starts from the official MINIX 3.3.0 release and implements all three requirements as reproducible experiments. Requirement 1 compares four CPU-scheduling policies using real child processes. Requirement 2 models configurable hierarchical page tables and compares FIFO with LRU replacement. Requirement 3 modifies the real MFS zone-allocation path to prefer contiguous free runs, then tests file and directory operations on a disposable MFS image.

Requirement	Implementation	Evidence
1. Scheduling	RR, SJF, priority, and MLFQ dispatcher with real workers	Known-workload PASS; 80 data rows
2. Paging	Sparse configurable hierarchy with FIFO/LRU frames	Known reference-string PASS; 72 data rows
3. MFS extents	Exact-origin bitmap allocation and free-run preference	I/O PASS; 18 rows; zero data errors

Scope and interpretation

The scheduling command controls logical slices but does not replace the production kernel scheduler. The paging command is a native MINIX simulator because x86 hardware page geometry cannot be changed from a configuration file. The MFS work is a real server modification: it changes allocation preference without changing the on-disk format. These boundaries keep the claims accurate.

2. How the project was completed

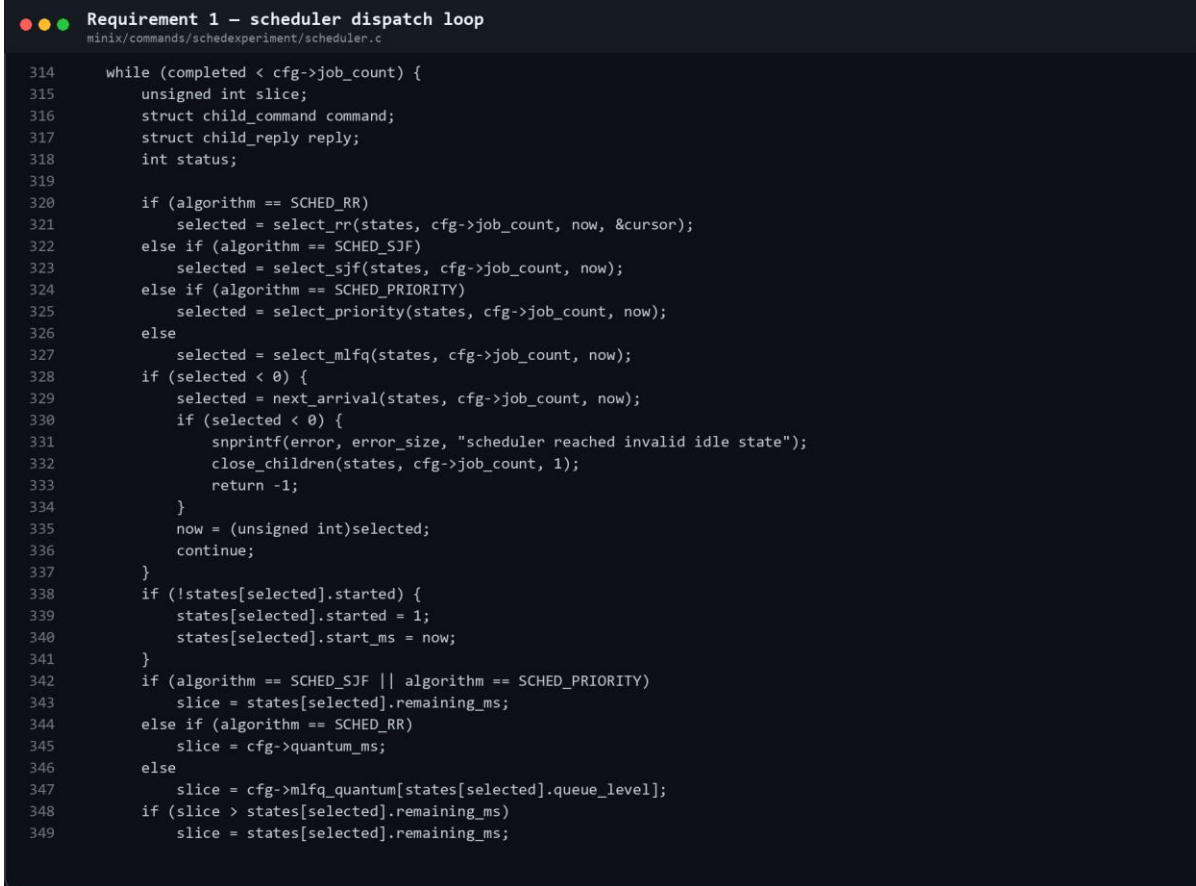
Stage	What was done
1. Clean baseline	The untouched 3.3.0 release was recorded so every modification is reviewable with git diff.
2. Implement	Three commands, parsers, matrix scripts, deterministic tests, and a focused MFS allocator change were added.
3. Transfer	The source was copied to <code>/usr/src</code> in the MINIX VM for the native compiler and headers.
4. Build	The commands and MFS server were compiled and installed from the submitted source.
5. Correctness gate	Known workloads ran first; a failed assertion or data mismatch would stop collection.
6. Experiment	Scripts varied one main parameter and wrote raw CSV. Extent tests remounted a disposable MFS image.
7. Preserve	CSV matrices, checksums, validation log, and console screenshots were copied into the deliverables.

Compilation and execution inside MINIX were the authoritative gate. Windows-side editing alone was never treated as proof, because only the VM has the correct compiler, headers, services, and runtime behavior.

3. Requirement 1 — scheduling

Implementation

The parent parses the workload, creates one child per job, and uses pipes to grant logical CPU slices. Each child performs deterministic integer work and acknowledges the slice. The parent selects only arrived jobs, records first start and completion, and calculates turnaround, waiting, response, makespan, and dispatch count. RR uses a rotating cursor, SJF selects the smallest remaining burst, priority selects the smallest numeric priority, and MLFQ uses increasing quanta with demotion.



```

314 while (completed < cfg->job_count) {
315     unsigned int slice;
316     struct child_command command;
317     struct child_reply reply;
318     int status;
319
320     if (algorithm == SCHED_RR)
321         selected = select_rr(states, cfg->job_count, now, &cursor);
322     else if (algorithm == SCHED_SJF)
323         selected = select_sjf(states, cfg->job_count, now);
324     else if (algorithm == SCHED_PRIORITY)
325         selected = select_priority(states, cfg->job_count, now);
326     else
327         selected = select_mlfq(states, cfg->job_count, now);
328     if (selected < 0) {
329         selected = next_arrival(states, cfg->job_count, now);
330         if (selected < 0) {
331             snprintf(error, error_size, "scheduler reached invalid idle state");
332             close_children(states, cfg->job_count, 1);
333             return -1;
334         }
335         now = (unsigned int)selected;
336         continue;
337     }
338     if (!states[selected].started) {
339         states[selected].started = 1;
340         states[selected].start_ms = now;
341     }
342     if (algorithm == SCHED_SJF || algorithm == SCHED_PRIORITY)
343         slice = states[selected].remaining_ms;
344     else if (algorithm == SCHED_RR)
345         slice = cfg->quantum_ms;
346     else
347         slice = cfg->mlfq_quantum[states[selected].queue_level];
348     if (slice > states[selected].remaining_ms)
349         slice = states[selected].remaining_ms;

```

Figure 1. Edited scheduler loop: policy selection, idle-time advance, and slice calculation.

Native result

Algorithm	Avg turnaround	Avg waiting	Avg response	Makespan	Dispatches
RR	185 ms	133 ms	17 ms	260 ms	14
SJF	137 ms	85 ms	85 ms	260 ms	5
Priority	155 ms	103 ms	103 ms	260 ms	5
MLFQ	191 ms	139 ms	5 ms	260 ms	15

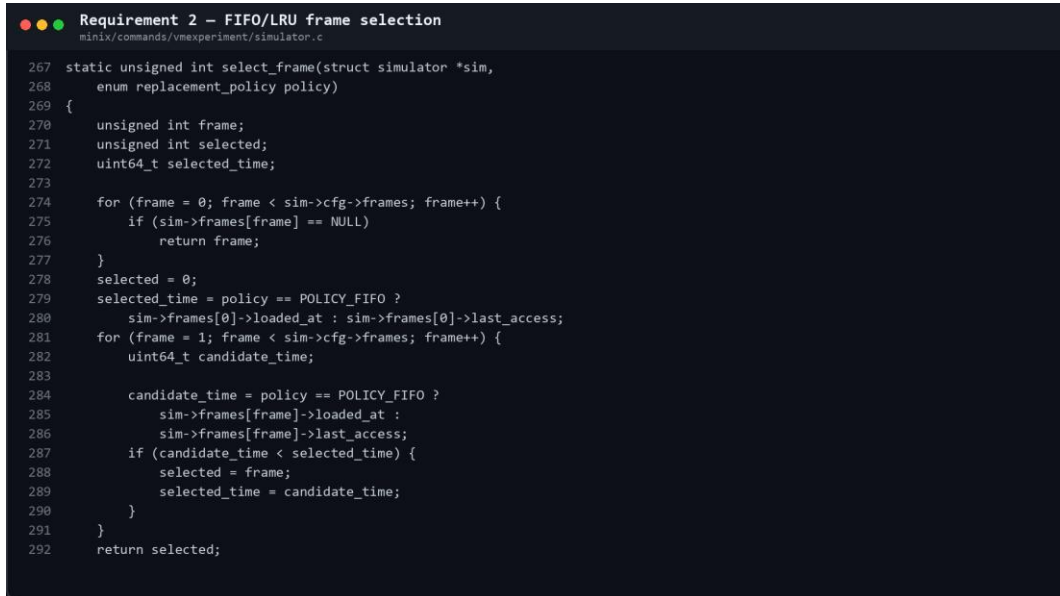
SJF produced the lowest average waiting and turnaround for this workload. MLFQ produced the quickest first response but required the most dispatches. Makespan is equal because every policy performs the same 260 logical milliseconds of work.

4. Requirement 2 — hierarchical paging

Implementation

The configuration specifies address width, page size, hierarchy levels and index widths, frame count, replacement policy, and deterministic trace parameters. Index bits plus page-offset bits must equal the address width. The sparse hierarchy creates only paths reached by the trace, allowing nodes, entries, and allocated bytes to be measured.

Every byte address is converted to a virtual page. A present page is a hit. On a fault, an unused frame is selected first; otherwise FIFO chooses the smallest load timestamp and LRU chooses the smallest last-access timestamp. First fills count as faults but not replacements.



```

267 static unsigned int select_frame(struct simulator *sim,
268     enum replacement_policy policy)
269 {
270     unsigned int frame;
271     unsigned int selected;
272     uint64_t selected_time;
273
274     for (frame = 0; frame < sim->cfg->frames; frame++) {
275         if (sim->frames[frame] == NULL)
276             return frame;
277     }
278     selected = 0;
279     selected_time = policy == POLICY_FIFO ?
280         sim->frames[0]->loaded_at : sim->frames[0]->last_access;
281     for (frame = 1; frame < sim->cfg->frames; frame++) {
282         uint64_t candidate_time;
283
284         candidate_time = policy == POLICY_FIFO ?
285             sim->frames[frame]->loaded_at :
286             sim->frames[frame]->last_access;
287         if (candidate_time < selected_time) {
288             selected = frame;
289             selected_time = candidate_time;
290         }
291     }
292     return selected;
  
```

Figure 2. Edited FIFO/LRU victim-selection logic.

Native result

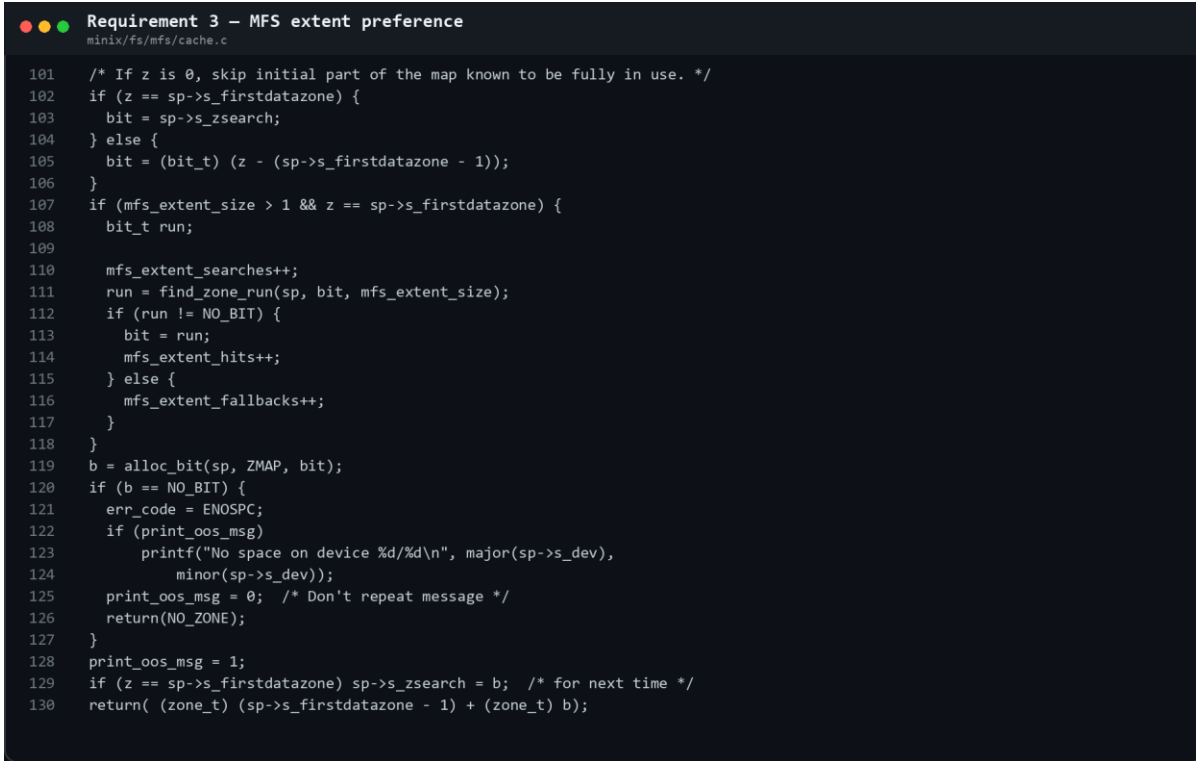
Page size	FIFO faults	LRU faults	Observed result
1,024 B	6,937	6,791	LRU lower by 146
2,048 B	4,735	4,047	LRU lower by 688
4,096 B	2,666	1,655	LRU lower by 1,011
8,192 B	1,378	1,064	LRU lower by 314

For the controlled locality trace with 64 frames, LRU faulted less than FIFO at every page size. Larger pages reduced faults because each frame covered more of the fixed byte working set; this does not imply that larger pages are universally better because fragmentation and transfer cost also increase.

5. Requirement 3 — extent-aware MFS

Implementation

Stock MFS allocates data zones through a bitmap. The change first scans for a free run matching `mfs_extent_size`. If found, its start becomes the exact origin passed to `alloc_bit`; otherwise the original behavior is retained and a fallback is counted. Only `alloc_bit` marks a zone allocated, so existing accounting and the on-disk format remain intact.



```

101  /* If z is 0, skip initial part of the map known to be fully in use. */
102  if (z == sp->s_firstdatazone) {
103      bit = sp->s_zsearch;
104  } else {
105      bit = (bit_t) (z - (sp->s_firstdatazone - 1));
106  }
107  if (mfs_extent_size > 1 && z == sp->s_firstdatazone) {
108      bit_t run;
109
110      mfs_extent_searches++;
111      run = find_zone_run(sp, bit, mfs_extent_size);
112      if (run != NO_BIT) {
113          bit = run;
114          mfs_extent_hits++;
115      } else {
116          mfs_extent_fallbacks++;
117      }
118  }
119  b = alloc_bit(sp, ZMAP, bit);
120  if (b == NO_BIT) {
121      err_code = ENOSPC;
122      if (print_oos_msg)
123          printf("No space on device %d/%d\n", major(sp->s_dev),
124              minor(sp->s_dev));
125      print_oos_msg = 0; /* Don't repeat message */
126      return(NO_ZONE);
127  }
128  print_oos_msg = 1;
129  if (z == sp->s_firstdatazone) sp->s_zsearch = b; /* for next time */
130  return( (zone_t) (sp->s_firstdatazone - 1) + (zone_t) b);

```

Figure 3. Edited MFS allocation path: preferred run, counters, and safe fallback.

Native result

Requested zones	Searches	Preferred-run hits	Fallbacks
1	0	0	0
2	6	6	0
4	6	6	0
8	6	6	0
16	6	6	0
32	6	6	0

The matrix used a disposable 64 MiB MFS image and produced 18 rows. Every file read matched its deterministic written pattern, so `verify_errors` stayed zero. All requested runs from 2 through 32 zones were found without fallback. Some short timings rounded to zero because of MINIX 3.3 timer resolution, so correctness and allocator counters are stronger evidence than tiny throughput differences.

6. Native MINIX build and test evidence

Commands executed in the VM

```
cd /usr/src && make includes
cd /usr/src/minix/commands/schedexperiment && make && make install
cd /usr/src/minix/commands/vmexperiment && make && make install
cd /usr/src/minix/commands/extentexperiment && make && make install
cd /usr/src/minix/fs/mfs && make && make install
cd /usr/src/minix/commands/schedexperiment/tests && sh test_known.sh
cd /usr/src/minix/commands/vmexperiment/tests && sh test_known.sh
cd /usr/src/minix/commands/extentexperiment/tests && sh test_known.sh
run_schedexperiments.sh /root/scheduling-matrix.csv
run_vmexperiments.sh /root/paging-matrix.csv
run_extent_matrix.sh /root/extent-matrix.csv
```

The known tests are deterministic oracles: expected scheduling averages are checked exactly, FIFO/LRU totals are compared with a known reference string, and extent I/O is read back and byte-verified. Matrix collection ran only after all three tests passed.

```
DETERMINISTIC_TESTS
PASS: known scheduling workload
PASS: known FIFO/LRU reference string
PASS: extent file/directory I/O and data verification
MATRIX_LINES
    81 /root/scheduling-matrix.csv
    73 /root/paging-matrix.csv
    19 /root/extent-matrix.csv
    173 total
MATRIX_CKSUM
3987883731 5385 /root/scheduling-matrix.csv
863209646 6398 /root/paging-matrix.csv
2664218402 1229 /root/extent-matrix.csv
MFS_EXTENT_LOGS
Aug 18 17:39:21 10 kernel: MFS: extent allocation preference is 1 zone(s)
Aug 18 17:39:21 10 kernel: MFS extent stats: size=1 searches=0 hits=0 fallbacks=
0
Aug 18 17:39:21 10 kernel: MFS: extent allocation preference is 2 zone(s)
Aug 18 17:39:21 10 kernel: MFS extent stats: size=2 searches=6 hits=6 fallbacks=
0
Aug 18 17:39:21 10 kernel: MFS: extent allocation preference is 4 zone(s)
Aug 18 17:39:21 10 kernel: MFS extent stats: size=4 searches=6 hits=6 fallbacks=
0
Aug 18 17:39:21 10 kernel: MFS: extent allocation preference is 8 zone(s)
# _
```

Figure 4. Actual VirtualBox MINIX console: PASS results, CSV row counts/checksums, and MFS counters.

MFS extent console output

```

3987883731 5385 /root/scheduling-matrix.csv
863209646 6398 /root/paging-matrix.csv
2664218402 1229 /root/extent-matrix.csv
MFS_EXTENT_LOGS
Aug 18 17:39:21 10 kernel: MFS: extent allocation preference is 1 zone(s)
Aug 18 17:39:21 10 kernel: MFS extent stats: size=1 searches=0 hits=0 fallbacks=
0
Aug 18 17:39:21 10 kernel: MFS: extent allocation preference is 2 zone(s)
Aug 18 17:39:21 10 kernel: MFS extent stats: size=2 searches=6 hits=6 fallbacks=
0
Aug 18 17:39:21 10 kernel: MFS: extent allocation preference is 4 zone(s)
Aug 18 17:39:21 10 kernel: MFS extent stats: size=4 searches=6 hits=6 fallbacks=
0
Aug 18 17:39:21 10 kernel: MFS: extent allocation preference is 8 zone(s)
Aug 18 17:39:21 10 kernel: MFS extent stats: size=8 searches=6 hits=6 fallbacks=
0
Aug 18 17:39:22 10 kernel: MFS: extent allocation preference is 16 zone(s)
Aug 18 17:39:22 10 kernel: MFS extent stats: size=16 searches=6 hits=6 fallbacks
=0
Aug 18 17:39:22 10 kernel: MFS: extent allocation preference is 32 zone(s)
Aug 18 17:39:22 10 kernel: MFS extent stats: size=32 searches=6 hits=6 fallbacks
=0
# _

```

Figure 5. Actual MINIX console: final extent-size counters recorded by MFS.

7. Changed files

Area	Main edited files	Purpose
Scheduling	minix/commands/schedexperiment/*	Parser, dispatcher, metrics, test, and matrix script
Paging	minix/commands/vmexperiment/*	Hierarchy, traces, FIFO/LRU, test, and matrix script
MFS	minix/fs/mfs/cache.c, super.c, main.c, mount.c	Exact bitmap origin, free-run scan, option, counters
Extent test	minix/commands/extentexperiment/*	Verified directory/file I/O and CSV metrics
Integration	minix/commands/Makefile; etc/Makefile; etc/*.conf	Build/install entries and configurations

8. Conclusion

All three requirements were built and executed in MINIX 3.3.0. The native correctness gate passed for scheduling, FIFO/LRU paging, and extent file/directory I/O. The matrices contain 80 scheduling rows, 72 paging rows, and 18 extent rows; their checksums are preserved. Requirement 1 demonstrates responsiveness versus dispatch cost, Requirement 2 shows lower LRU faults for the chosen locality workload, and Requirement 3 confirms successful contiguous-run preference with safe fallback and correct data.

Limitations are explicit: scheduler dispatches are experiment events rather than kernel context-switch counters; paging faults are simulated rather than hardware telemetry; and coarse MINIX timing weakens very short throughput measurements. These do not invalidate deterministic correctness or configuration-controlled comparisons.

Submission evidence

- Raw results: project-deliverables/native-results/*.csv and native-validation.txt
- Source comparison: git diff baseline-v3.3.0..HEAD
- Report builder: tools/create_cse335_report.py